

# Cahier des Charges (DSF)

## *Mini Garage - Application de gestion de garage automobile*

### 1. Introduction

Ce document a pour but de définir les spécifications fonctionnelles, techniques et les exigences générales pour le développement de l'application web « Mini Garage ».

Mini Garage est une application de gestion destinée à un garage automobile. Elle permet au personnel du garage de suivre de façon centralisée les véhicules pris en charge, les réparations effectuées sur ces véhicules et les techniciens qui interviennent sur chaque réparation.

L'application est développée en utilisant **React** pour la partie **frontend** (interface utilisateur) et **Laravel** pour la partie **backend** (API REST et gestion des données en base).

### 2. Objectifs du Projet

L'application répond aux besoins suivants :

- Remplacer un suivi papier ou tableur du parc de véhicules et des réparations par une interface web centralisée et structurée.
- Permettre de retrouver rapidement l'historique des réparations d'un véhicule donné.
- Permettre de savoir quels techniciens sont intervenus sur quelle réparation.
- Simplifier la recherche d'un véhicule par sa marque ou son immatriculation.

Objectifs pédagogiques associés :

- Mettre en pratique le développement full-stack avec React et Laravel.
- Manipuler des relations de données réalistes : un-à-plusieurs (véhicule → réparations) et plusieurs-à-plusieurs (réparations ↔ techniciens).
- Construire une API REST propre avec validations côté serveur.

### 3. Portée du Projet

#### Fonctionnalités incluses dans cette version

- Gestion complète (CRUD) des véhicules.
- Gestion complète (CRUD) des techniciens.
- Gestion complète (CRUD) des réparations, avec association à un véhicule et à un ou plusieurs techniciens.
- Recherche et filtrage des véhicules par marque ou immatriculation.
- Consultation de l'historique des réparations d'un véhicule

Authentification de l'administrateur (connexion par email / mot de passe, token API Laravel Sanctum) protégeant l'accès à l'ensemble de l'application.

#### Fonctionnalités explicitement exclues de cette version

- Facturation et gestion des paiements.
- Gestion des stocks de pièces détachées.
- Notifications automatiques (rappels d'entretien, alertes).

- Tableau de bord statistique avancé.

### Utilisateurs types

Dans cette version, l'application ne gère qu'un seul profil d'utilisateur implicite : le personnel du garage (réceptionniste ou gérant), qui consulte et met à jour les données. Aucune distinction de rôles ni d'authentification n'est mise en place à ce stade.

## 4. Exigences Fonctionnelles

### Module Véhicules

- Consulter la liste de tous les véhicules, avec recherche par marque ou immatriculation.
- Basculer entre un affichage en tableau et un affichage en cartes.
- Consulter la fiche détaillée d'un véhicule, avec l'historique de ses réparations.
- Créer un nouveau véhicule (immatriculation, marque, modèle, couleur, année, kilométrage, carrosserie, énergie, boîte de vitesses).
- Modifier les informations d'un véhicule existant.
- Supprimer un véhicule (ce qui supprime en cascade ses réparations associées).

### Module Techniciens

- Consulter la liste des techniciens.
- Consulter la fiche d'un technicien avec les réparations sur lesquelles il est intervenu.
- Créer, modifier et supprimer un technicien (nom, prénom, spécialité).

### Module Réparations

- Consulter la liste des réparations, avec le véhicule et les techniciens associés.
- Consulter le détail d'une réparation.
- Créer une réparation : choix du véhicule concerné, date, durée de main d'œuvre, objet de la réparation, et sélection d'un ou plusieurs techniciens.
- Modifier une réparation existante (y compris les techniciens assignés).
- Supprimer une réparation.

## 5. Modèle de Données / Structure des Données

Trois entités principales structurent l'application : Véhicule, Réparation et Technicien.

**Véhicule** : immatriculation, marque, modèle, couleur, année, kilométrage, carrosserie, énergie, boîte de vitesses.

**Réparation** : véhicule concerné, date, durée de main d'œuvre, objet de la réparation.

**Technicien** : nom, prénom, spécialité.

Relations entre les entités :

- Un Véhicule peut avoir plusieurs Réparations (relation un-à-plusieurs).
- Une Réparation appartient à un seul Véhicule.
- Une Réparation peut être assignée à plusieurs Techniciens, et un Technicien peut intervenir sur plusieurs Réparations (relation plusieurs-à-plusieurs, via une table de liaison).

Ces informations guideront directement la création des migrations Laravel (consulter le Dossier de Conception Technique pour le détail des colonnes et des types).

## 6. Exigences Non Fonctionnelles

**Performance** : les listes (véhicules, réparations, techniciens) doivent s'afficher rapidement. Les relations (réparations d'un véhicule, techniciens d'une réparation) sont chargées en une seule requête grâce au eager loading Laravel (with()), afin d'éviter le problème des requêtes N+1.

**Sécurité** : l'accès à l'application est protégé par une authentification (Laravel Sanctum, token API) - seul l'administrateur connecté peut consulter et modifier les données. Toutes les données envoyées à l'API sont en plus validées côté serveur (règles Laravel : champs requis, types, longueurs, unicité de l'immatriculation, existence des identifiants liés). Les suppressions en cascade sont gérées au niveau de la base de données pour garantir la cohérence.

**Convivialité / Ergonomie** : interface simple et cohérente basée sur Bootstrap, navigation par menu explicite (Véhicules, Réparations, Techniciens), messages de confirmation avant suppression.

**Compatibilité** : application responsive (grille Bootstrap), utilisable sur les navigateurs modernes (Chrome, Firefox, Edge) ainsi que sur mobile et tablette.

**Scalabilité** : l'architecture découplée frontend / API REST permet de faire évoluer indépendamment les deux parties, et de remplacer SQLite par MySQL ou PostgreSQL sans changer le code métier.

**Accessibilité** : champs de formulaire correctement étiquetés, structure HTML sémantique, icônes toujours accompagnées d'un libellé texte.

**Éco-responsabilité** : les appels API ne renvoient que les données nécessaires à chaque écran, les relations sont chargées en une seule requête plutôt qu'en plusieurs appels successifs, et l'interface reste sobre (pas d'images lourdes ni d'animations superflues).

## 7. Architecture Technique

**Architecture globale** : application web monopage (SPA) côté client, communiquant avec une API REST côté serveur (architecture client-serveur découplée).

**Technologies Frontend** : React 19, React Router DOM (navigation), Bootstrap 5 et Bootstrap Icons (mise en forme), Vite (outil de build et serveur de développement).

**Technologies Backend** : Laravel 13 (PHP 8.4), Laravel Sanctum pour l'authentification par token API, base de données SQLite en développement / PostgreSQL en production, API REST exposée via des ressources Laravel (Route::apiResource), conteneurisée avec Docker pour le déploiement.

**Communication Frontend / Backend** : requêtes HTTP au format JSON via l'API Fetch native de JavaScript, en direction des endpoints Laravel (ex. /api/vehicules). L'URL de l'API est paramétrée via la variable d'environnement VITE\_BACKEND\_URL.

**Environnement de développement** : serveur de développement Vite pour le frontend, php artisan serve pour le backend.

## 8. Interface Utilisateur et Expérience Utilisateur (UI/UX)

**Design** : style sobre et fonctionnel, thème « garage » avec une navbar de couleur foncée et des icônes Bootstrap Icons évoquant l'automobile et la mécanique (voiture, clé à molette, personnes).

**Navigation** : gérée par react-router-dom ; une barre de navigation persistante permet d'accéder aux trois modules (Véhicules, Réparations, Techniciens) depuis n'importe quelle page.

**Composants clés** : barre de navigation (Navbar), cartes de contenu réutilisables, tableaux de listing, formulaires avec validation et messages d'erreur.

Le détail des écrans (wireframes) et des parcours utilisateurs est présenté dans le Dossier de Conception Visuelle qui accompagne ce document.

## 9. Flux de Navigation

**Scénario « Ajouter une réparation » :** l'utilisateur se rend sur la page Réparations, clique sur « Nouvelle réparation », sélectionne le véhicule concerné et un ou plusieurs techniciens, renseigne la date, la durée de main d'œuvre et l'objet de la réparation, puis clique sur « Enregistrer ». Il est ensuite redirigé vers la liste des réparations.

**Scénario « Rechercher un véhicule » :** l'utilisateur accède à la page Véhicules et saisit une marque ou une immatriculation dans le champ de recherche ; la liste se met à jour dynamiquement.

**Scénario « Consulter l'historique d'un véhicule » :** depuis la liste des véhicules, l'utilisateur clique sur un véhicule et accède à sa fiche détaillée, qui affiche l'ensemble des réparations liées à ce véhicule.

Ces parcours sont illustrés sous forme de schémas dans le Dossier de Conception Visuelle.

## 10. Échéancier et Plan de Travail

### Semaine 5 - Conception

- Jours 1-2 : Choix et validation du sujet (fait - projet Mini Garage déjà entamé).
- Jour 3 : Rédaction du présent cahier des charges (fait).
- Jour 4 : Relecture et ajustements du cahier des charges (fait).
- Jours 5-7 : Dossier de conception visuelle (wireframes, flux) et dossier de conception technique (modèle de données, endpoints API, structure React) (fait).

### Semaine 6 - Réalisation

- Jours 8-9 : Consolidation du frontend React (pages, composants, navigation, appels API) - fait.
- Jours 10-11 : Consolidation du backend Laravel (migrations, modèles, contrôleurs, routes API) - fait.
- Jour 12 : Vérification de l'intégration front/back et de la gestion des erreurs (fait).
- Jour 13 : Tests backend (PHPUnit/Pest) et vérifications fonctionnelles côté front (fait).
- Jour 14 : Optimisation, sécurité et éco-responsabilité (fait).
- Jour 15 : Dépôt GitHub, rédaction du README, préparation de la démonstration (fait).

## 11. Plan de Validation et Tests

**Tests backend :** tests Laravel (PHPUnit/Pest) sur les endpoints API (création, modification, suppression) et sur les règles de validation (ex. immatriculation obligatoire et unique).

**Tests fonctionnels :** vérification manuelle des parcours utilisateurs complets décrits en section 9 (création d'une réparation, recherche d'un véhicule, consultation d'un historique).

**Stratégie de test :** combinaison de tests automatisés ciblés côté API et de scénarios de vérification manuelle côté interface React.

**Validation du DSF :** chaque fonctionnalité listée en section 4 est vérifiée individuellement avant la démonstration finale.

## 12. Déploiement et Environnement d'Exécution

**Environnement de démonstration** : application déployée publiquement - backend (API Laravel) hébergé sur Render, frontend (React) hébergé sur Netlify. Les deux services communiquent via l'API REST, avec CORS ouvert sur les routes /api/\*.

**Prérequis techniques** : Node.js et npm (frontend), PHP 8.4, Composer, extension SQLite (dev) ou PostgreSQL (production), Docker pour le déploiement du backend.

**Liens de démonstration** :

**Application (frontend)** : <https://mini-garage.netlify.app/>

**API (backend)** : <https://mini-garage-api-o7m5.onrender.com/>

**Présentation vidéo** : [Lien vers la présentation](#)

## 13. Références

- Documentation officielle Laravel : <https://laravel.com/docs>
- Documentation officielle React : <https://react.dev>
- Documentation React Router : <https://reactrouter.com>
- Documentation Bootstrap 5 : <https://getbootstrap.com>
- Création de wireframes : <https://www.youtube.com/watch?v=zlooD7F5NKA>
- Laravel Sanctum + Register and Login : <https://www.youtube.com/watch?v=ajUST-jUMeg>